

PUC-Rio

INF1040 - Projeto de Programação Modular

**Trabalho T2 - Projeto da Aplicação
BackupManager**

Grupo 6

Caio Alexandre da Silva Gonçalves
Davi Teixeira Mendes Almeida

Rio de Janeiro
2026

Sumario:

1. Definição do Tema - Descrição da Aplicação	3
1.1 Escopo do sistema	3
2. Entidades e Funcionalidades	3
2.1 Estrutura do perfil de backup	4
2.2 Funcionalidades levantadas	4
3. Interfaces e Diagrama	5
3.1 Interface	6
3.2 Estrutura física do projeto atual	6
3.3 Responsabilidade dos módulos	7
4. Relacionamento Cliente-Servidor	7
4.1 Fluxo de criação de perfil	7
4.2 Fluxo de execução de backup	8
5. Funções de Acesso	8
6. Codigos de Retorno	9
7. Casos de Teste Automatizados	9
7.1 Lista de casos de teste planejados	10
8. Considerações de Modularização	10
9. Conclusão	10

1. Definição do Tema - Descrição da Aplicação

O **BackupManager** é uma aplicação desktop local voltada para o gerenciamento de rotinas de backup entre diretórios do próprio computador ou de discos conectados. A aplicação permite que o usuário crie diferentes perfis de backup, cada um com suas próprias origens, destinos, restrições, operação e forma de execução.

O objetivo principal do sistema é simplificar a configuração de cópias de segurança sem exigir que o usuário escreva comandos manualmente. Cada perfil funciona como uma rotina independente, permitindo organizar backups para diferentes finalidades, como documentos da faculdade, projetos de programação, arquivos pessoais, imagens, materiais de RPG ou cópias destinadas a um disco externo.

A aplicação foi projetada respeitando as restrições do trabalho: uso exclusivo de Python, uso apenas de bibliotecas padrão, ausência de classes, ausência de orientação a objetos e representação das entidades por meio de dicionários. Essa restrição reforça a necessidade de uma boa divisão modular, pois a organização do sistema passa a depender da separação clara entre módulos e funções.

1.1 Escopo do sistema

- Criar, consultar, alterar, ativar, desativar e excluir perfis de backup.
- Associar múltiplas pastas de origem a um mesmo perfil.
- Associar múltiplos destinos a um mesmo perfil.
- Permitir operação de cópia ou movimentação de arquivos.
- Filtrar arquivos por extensão, trecho no nome, tamanho e data de modificação.
- Executar backup manualmente.
- Preparar a estrutura para execução automática por intervalo ou por alteração.
- Salvar perfis, configurações e histórico em arquivos JSON.
- Registrar códigos de retorno e mensagens padronizadas.
- Validar funcionalidades por meio de testes automatizados com unittest.

2. Entidades e Funcionalidades

Como a aplicação não utiliza classes, as entidades do sistema são representadas por dicionários. Cada dicionário possui campos padronizados e é manipulado por funções específicas dentro dos módulos do sistema.

Entidade	Representacao	Descricao
Perfil de backup	dict	Define uma rotina completa de backup, com nome, origens, destinos, restrições, agendamento e estado.
Restrições	dict	Define filtros aplicados aos arquivos antes da execução do backup.
Agendamento	dict	Define se o backup sera manual, por intervalo ou por alteração.

Historico	dict/list	Registra execuções realizadas, status, quantidade de arquivos e erros.
Arquivo analisado	dict	Representa metadados de um arquivo encontrado em uma origem.
Estado global	dict	Mantém perfis, histórico e configurações carregados em memória.

2.1 Estrutura do perfil de backup

```

perfil = {
    "id": "perfil_001",
    "nome": "Backup Faculdade",
    "origens": [],
    "destinos": [],
    "operacao": "copiar",
    "restricoes": {
        "extensoes_permitidas": [],
        "nome_contem": "",
        "tamanho_min": 0,
        "tamanho_max": None,
        "data_modificacao_min": None,
        "data_modificacao_max": None
    },
    "agendamento": {
        "tipo": "manual",
        "intervalo_minutos": None,
        "executar_ao_detectar_mudanca": False,
        "ultima_execucao": None
    },
    "estado_arquivos": {},
    "ativo": True
}

```

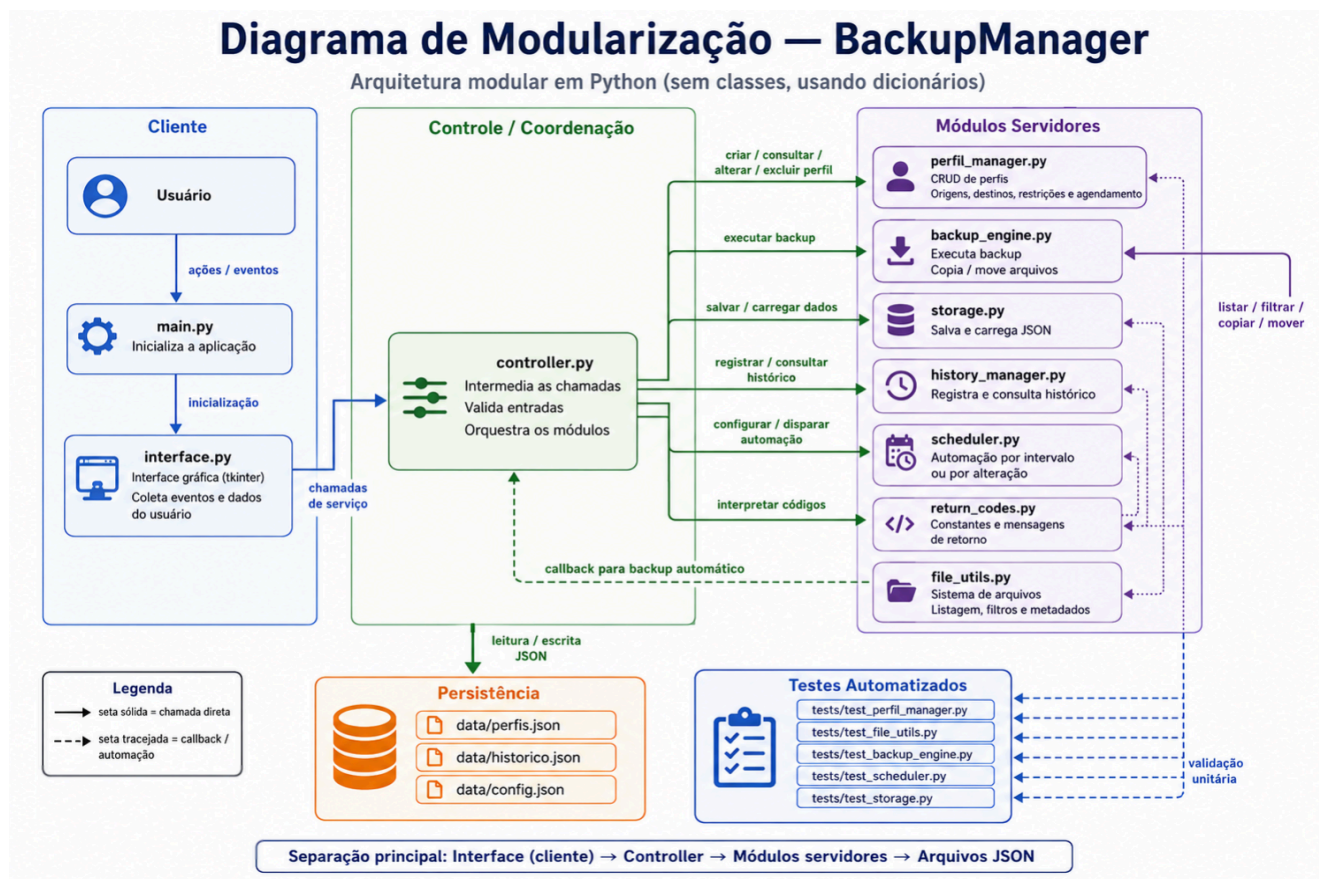
2.2 Funcionalidades levantadas

Grupo	Funcionalidades
Perfis	Criar, listar, consultar, alterar nome, excluir, ativar e desativar perfis.
Origins	Adicionar e remover diretórios de origem.
Destinos	Adicionar e remover diretórios de destino.
Operacao	Definir se o perfil copia ou move arquivos.
Restrictors	Configurar filtros por extensão, nome, tamanho e data de modificação.
Backup	Executar rotina de backup associada a um perfil.
Agendamento	Verificar execução por intervalo ou por alteração de arquivos.
Historico	Registrar e consultar execuções realizadas.
Armazenamento	Salvar e carregar dados em JSON.
Testes	Validar funções principais com unittest.

3. Interfaces e Diagrama

O diagrama apresenta a divisão modular do BackupManager. A interface atua como cliente, recebendo as ações do usuário e enviando as solicitações ao **controller.py**, que centraliza a coordenação do sistema. A partir dele, são chamados os módulos servidores, responsáveis por funções específicas como gerenciar perfis, executar backups, acessar arquivos, salvar dados, registrar histórico e controlar agendamentos.

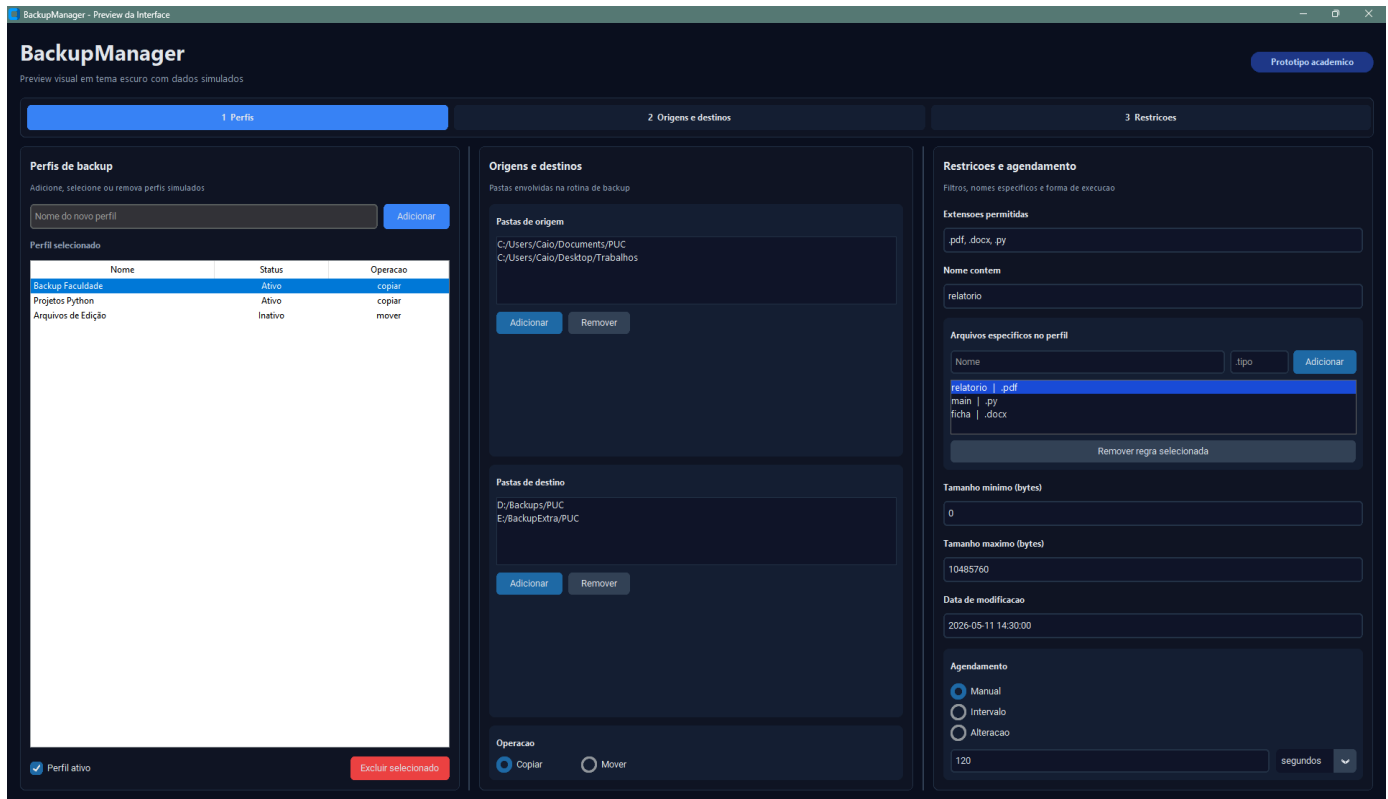
As setas sólidas representam chamadas diretas entre módulos, enquanto as tracejadas indicam callbacks ou automações, como a execução automática de backups. A persistência dos dados é feita em arquivos JSON, e os testes automatizados validam separadamente os principais módulos do sistema.



3.1 Interface

A interface prevista para o BackupManager é uma tela única, criada com customTkinter, dividida em três áreas principais. A primeira área concentra a lista de perfis; a segunda área apresenta origens e destinos; a terceira área agrupa restrições e configurações de agendamento.

Prototipo:



3.2 Estrutura física do projeto atual

```
- backupmanager/  
| |-- __init__.py  
| |-- main.py  
| |-- interface.py  
| |-- controller.py  
| |-- perfil_manager.py  
| |-- backup_engine.py  
| |-- file_utils.py  
| |-- scheduler.py  
| |-- storage.py  
| |-- history_manager.py  
| |-- return_codes.py  
|  
- data/  
| |-- perfis.json  
| |-- historico.json  
| |-- config.json  
|  
- tests/  
| |-- test_backup_engine.py  
| |-- test_file_utils.py  
| |-- test_perfil_manager.py  
| |-- test_scheduler.py  
| |-- test_storage.py
```

3.3 Responsabilidade dos módulos

Modulo	Responsabilidade principal
main.py	Inicia a aplicação chamando a interface principal.
interface.py	Cria a tela com customtkinter e recebe interações do usuário.
controller.py	Intermedia a interface e os módulos internos.
perfil_manager.py	Gerência criação, consulta e alteração de perfis.
backup_engine.py	Executa a lógica de backup e monta resultados.
file_utils.py	Fornecer funções auxiliares para arquivos, diretórios e filtros.
scheduler.py	Controla verificações para execução automática.
storage.py	Salva e carrega dados em arquivos JSON.
history_manager.py	Cria, registra e consulta histórico de backups.
return_codes.py	Centraliza códigos de retorno e mensagens do sistema.

4. Relacionamento Cliente-Servidor

O módulo que solicita uma operação atua como cliente; o módulo que oferece a funcionalidade atua como servidor.

```
Usuario
-> interface.py
   -> controller.py
       -> perfil_manager.py
       -> backup_engine.py
           -> file_utils.py
       -> scheduler.py
           -> file_utils.py
       -> storage.py
       -> history_manager.py
```

4.1 Fluxo de criação de perfil

1. Usuario solicita a criação de um perfil na interface.
2. interface.py coleta o nome informado.
3. interface.py chama controller.criar_novo_perfil(nome).
4. controller.py chama perfil_manager.criar_perfil(nome).
5. perfil_manager.py valida o nome e cria o dicionário do perfil.
6. controller.py adiciona o perfil ao estado global.
7. controller.py chama storage.salvar_perfis(perfis).
8. interface.py atualiza a lista visual de perfis.

4.2 Fluxo de execução de backup

1. Usuário solicita a execução manual de um perfil.
2. interface.py chama controller.executar_backup_do_perfil(perfil_id).
3. controller.py consulta o perfil em perfil_manager.py.

4. controller.py chama backup_engine.executar_backup(perfil).
5. backup_engine.py usa file_utils.py para listar e filtrar arquivos.
6. backup_engine.py copia ou move arquivos para os destinos.
7. backup_engine.py retorna um resultado.
8. controller.py registra o resultado no history_manager.py.
9. controller.py salva o historico com storage.py.
10. interface.py exibe mensagem ao usuário.

5. Funções de Acesso

As funções de acesso representam as operações públicas oferecidas por cada módulo. Elas reduzem o acoplamento porque outros módulos não precisam conhecer a implementação interna, apenas a interface funcional disponível.

Modulo	Funcoes principais
controller.py	inicializar_aplicacao, criar_novo_perfil, obter_perfis, obter_perfil_por_id, salvar_perfil_editado, excluir_perfil_por_id, adicionar_origem_ao_perfil, adicionar_destino_ao_perfil, definir_restricoes_do_perfil, definir_agendamento_do_perfil, executar_backup_do_perfil, consultar_historico_do_perfil
perfil_manager.py	criar_perfil, consultar_perfil, listar_perfis, alterar_nome_perfil, excluir_perfil, ativar_perfil, desativar_perfil, adicionar_origem, remover_origem, adicionar_destino, remover_destino, alterar_operacao, alterar_restricoes, alterar_agendamento
backup_engine.py	executar_backup, executar_backup_multiplos_destinos, processar_arquivo_para_destinos, copiar_arquivo, mover_arquivo, gerar_caminho_destino, criar_pasta_destino_se_necessario, montar_resultado_backup
file_utils.py	caminho_existe, caminho_e_diretorio, listar_arquivos_em_origem, listar_arquivos_de_origens, obter_metadados_arquivo, arquivo_atende_restricoes, verificar_permissao_leitura, verificar_permissao_escrita
scheduler.py	deve_executar, deve_executar_por_intervalo, deve_executar_por_alteracao, obter_estado_atual_arquivos, comparar_estado_arquivos, atualizar_estado_arquivos, iniciar_monitoramento, parar_monitoramento
storage.py	salvar_json, carregar_json, salvar_perfis, carregar_perfis, salvar_historico, carregar_historico, salvar_configuracoes, carregar_configuracoes
history_manager.py	criar_registro_historico, registrar_backup, consultar_historico_por_perfil, listar_historico, limpar_historico_perfil, limpar_todo_historico
return_codes.py	obter_mensagem

6. Codigos de Retorno

O sistema utiliza códigos de retorno para padronizar o resultado das operações. Esse padrão evita que os módulos dependam apenas de mensagens impressas ou exceções não tratadas. Quando uma função precisa retornar dados e status, ela utiliza uma tupla no formato código, valor.

```
codigo, perfil = consultar_perfil(perfis, perfil_id)

if codigo != OK:
    return codigo, None

return OK, perfil
```

Codigo	Constante	Significado
0	OK	Operação realizada com sucesso.
1	ERRO_PERFIL_NAO_ENCONTRADO	O perfil solicitado não existe.
2	ERRO_NOME_INVALIDO	O nome informado é vazio ou invalido.
3	ERRO_ORIGEM_INVALIDA	A origem informada é inválida.
4	ERRO_DESTINO_INVALIDO	O destino informado e invalido.
5	ERRO_SEM_PERMISSAO	O sistema não possui permissão para acessar o caminho.
6	ERRO_ARQUIVO_NAO_ENCONTRADO	O arquivo esperado não foi encontrado.
7	ERRO_RESTRICAO_INVALIDA	Restricao informada e invalida.
8	ERRO_OPERACAO_INVALIDA	Operação diferente de copiar ou mover.
9	ERRO_AGENDAMENTO_INVALIDO	Agendamento incorreto ou incompleto.
10	ERRO_FALHA_AO_COPIAR	Falha ao copiar arquivo.
11	ERRO_FALHA_AO_MOVER	Falha ao mover arquivo.
12	ERRO_JSON_CORROMPIDO	Arquivo JSON não pode ser interpretado.
13	ERRO_BACKUP_SEM_ARQUIVOS	Nenhum arquivo atende às condições do backup.
14	ERRO_DESTINO_SEM_ESPACO	Destino sem espaço disponível.
15	ERRO_PERFIL_INATIVO	O perfil está inativo.
16	ERRO_DADOS_INVALIDOS	Os dados recebidos são inválidos.

7. Casos de Teste Automatizados

Os testes automatizados foram criados com unittest, biblioteca padrão do Python. A base atual do projeto possui testes para os módulos de perfis, filtros de arquivos, armazenamento, backup e agendamento.

Arquivo de teste	O que valida
test_perfil_manager.py	Criação de perfil válido, rejeição de nome vazio, consulta de perfil existente e inexistente.
test_file_utils.py	Filtros por extensão, nome e tamanho mínimo.
test_backup_engine.py	Montagem do resultado de backup e comportamento inicial sem arquivos.
test_scheduler.py	Comparação de estado de arquivos e parada de monitoramento.
test_storage.py	Salvar/carregar JSON, JSON inexistente e JSON corrompido.

7.1 Lista de casos de teste planejados

Caso	Entrada / Condicao	Resultado esperado
Criar perfil valido	Nome preenchido	Retorna OK e cria dicionário do perfil.
Criar perfil invalido	Nome vazio	Retorna ERRO_NOME_INVALIDO.
Consultar perfil existente	ID presente na lista	Retorna OK e o perfil.
Consultar perfil inexistente	ID ausente	Retorna ERRO_PERFIL_NAO_ENCONTRADO.
Adicionar origem	Caminho informado	Origem inserida no perfil.
Adicionar destino	Caminho informado	Destino inserido no perfil.
Alterar operacao	copiar ou mover	Operacao salva no perfil.
Rejeitar operacao	Valor diferente de copiar/mover	Retorna ERRO_OPERACAO_INVALIDA.
Filtrar extensao	Arquivo .py e restrição .py	Arquivo aceito.
Filtrar nome	Nome contém trecho definido	Arquivo aceito.
Filtrar tamanho	Tamanho dentro dos limites	Arquivo aceito.
Salvar perfis	Lista de perfis	Arquivo JSON e criado ou atualizado.
Carregar JSON corrompido	JSON invalido	Retorna ERRO_JSON_CORROMPIDO.
Registrar historico	Resultado de backup	Registro adicionado ao historico.
Comparar alteracao	Estado antigo diferente do novo	Detecta necessidade de backup.

8. Considerações de Modularização

A estrutura do BackupManager busca manter alta coesão dentro de cada módulo. O módulo de perfis manipula perfis; o módulo de armazenamento manipula JSON; o módulo de arquivos concentra operações auxiliares do sistema de arquivos; o módulo de interface apenas interage com o usuário; e o controlador organiza as chamadas entre as partes.

Essa divisão também reduz o acoplamento, pois a interface não precisa conhecer detalhes da execução do backup, do salvamento em JSON ou da consulta ao histórico. Ela chama o controlador, que decide qual módulo deve resolver cada tarefa. A existência de códigos de retorno padronizados também melhora a comunicação entre os módulos.

Como o projeto não pode usar classes, os dicionários funcionam como estruturas compartilhadas entre as funções. Para evitar perda de organização, os campos desses dicionários devem ser mantidos padronizados e documentados.

9. Conclusão

O BackupManager atende a proposta de uma aplicação modular para gerenciamento de backups locais. O sistema foi planejado de forma a separar interface, controle, regras de negócio, acesso a arquivos, armazenamento, histórico e agendamento.

A decisão de representar entidades por dicionários, em vez de classes, segue a restrição definida para o projeto e reforça a importância da documentação das estruturas de dados. Mesmo sem orientação a objetos, a aplicação mantém uma organização clara por módulos e funções de acesso.